

Engenharia de Software e Desenvolvimento

Aula 9 - Padrões de projeto e ferramentas CASE

Campo	Definição da aula
Disciplina	Engenharia de Software e Desenvolvimento - 10 questões no bloco específico
Aula	9 de 17 - Padrões de projeto e ferramentas CASE
Itens do edital cobertos	1.9 Padrões de projeto; 1.11 Ferramentas CASE
Objetivo	Distinguir padrões GoF, reconhecer intenção de cada padrão, separar padrão de projeto, padrão arquitetural e ferramenta CASE, e resolver questões FGV com alternativas próximas.
Perfil da banca	FGV costuma cobrar descrição curta de padrão, associação padrão-finalidade, caso de sistema corporativo, comparação entre padrões vizinhos e armadilhas de terminologia.

Nota editorial. Esta aula é autoral. As questões são inéditas. Provas FGV foram consultadas apenas para calibrar formato, densidade e nível de cobrança, sem cópia literal de itens reais.

1. Recorte exato da aula e prioridade para a prova

Esta aula cobre dois tópicos pontuais da trilha: padrões de projeto e ferramentas CASE - Computer-Aided Software Engineering. O foco não é ensinar todos os padrões existentes em profundidade enciclopédica, mas entregar o que mais aparece em prova: intenção, categoria, estrutura conceitual, uso típico, confusão com padrões vizinhos e efeito prático em sistemas corporativos.

No edital do TJSC, padrões de projeto aparecem em Engenharia de Software e Desenvolvimento, enquanto padrões arquiteturais como MVC - Model-View-Controller e DDD - Domain-Driven Design também aparecem em outra disciplina, Arquitetura de Sistemas e Integração. Nesta aula, MVC e DDD serão citados apenas para evitar confusão; o núcleo aqui são padrões de projeto em nível de classe/objeto e ferramentas CASE.

Meta de prova: se a FGV descrever uma situação de criação de objetos, adaptação de interfaces, notificação de interessados, variação de algoritmo, adição dinâmica de comportamento, ponto único de acesso ou suporte automatizado à modelagem, você deve reconhecer o conceito correto sem depender de decorar nomes isolados.

1.1 Como a FGV vem cobrando este assunto

Nos cadernos oficiais recentes de TI, a FGV cobra padrões de projeto de forma direta, mas com alternativas muito próximas. Exemplos de estrutura observada: relacionar descrições como “notifica automaticamente dependentes”, “encapsula famílias de algoritmos”, “garante uma única instância”, “subclasses decidem qual classe concreta instanciar” e “adiciona responsabilidades dinamicamente”; ou perguntar qual padrão é responsável pela criação de objetos mediante especialização da classe principal.

Tradução para o treino: as questões desta aula não vão perguntar apenas “o que é Singleton?”. Elas vão colocar um módulo de tramitação, uma API de pagamentos, uma política de autorização, um gerador de documentos ou um editor de modelos e exigir a distinção entre padrões parecidos.

2. Padrões de projeto: visão geral

Padrão de projeto é uma solução reutilizável, nomeada e recorrente para um problema comum de projeto de software em determinado contexto. Ele não é código pronto, não é biblioteca obrigatória, não é arquitetura completa e não substitui análise do problema.

A formulação clássica mais cobrada em concursos vem do livro Design Patterns: Elements of Reusable Object-Oriented Software, associado aos autores Erich Gamma, Richard Helm, Ralph Johnson e John Vlissides, conhecidos como GoF - Gang of Four. A banca costuma cobrar a divisão em três famílias: criacionais, estruturais e comportamentais.

Família	Pergunta que responde	Ideia central	Exemplos mais cobrados
Criacionais	Como criar objetos?	Abstraem ou controlam a criação de objetos, reduzindo acoplamento com classes concretas.	Factory Method, Abstract Factory, Builder, Prototype, Singleton
Estruturais	Como compor classes/objetos?	Organizam a composição entre objetos e interfaces, facilitando adaptação, extensão, proteção ou simplificação.	Adapter, Facade, Decorator, Proxy, Composite, Bridge, Flyweight
Comportamentais	Como objetos colaboram e distribuem responsabilidades?	Organizam algoritmos, comunicação, estados, comandos e variações de comportamento.	Strategy, Observer, State, Command, Template Method, Iterator, Mediator, Chain of Responsibility, Visitor, Memento

Pegadinha FGV. A banca pode usar uma definição verdadeira de um padrão no contexto de outro. Exemplo: “encapsula criação” pode lembrar Factory Method, mas se a questão falar em “famílias de objetos relacionados”, o candidato deve pensar em Abstract Factory.

3. Padrões criacionais

Padrões criacionais lidam com a criação de objetos. Eles são úteis quando o código cliente não deve depender diretamente de classes concretas, construtores específicos ou combinações complexas de objetos. Em sistemas Java/Quarkus, isso aparece em factories, producers, builders de DTOs, geração de clientes para integrações e criação controlada de serviços.

3.1 Singleton

Singleton garante que **uma classe tenha uma única instância e oferece um ponto global/controlado de acesso a ela**. Em prova, as palavras-chave são: única instância, acesso global, instância compartilhada e controle de criação.

Exemplo conceitual: um provedor de configuração de ambiente carregado uma única vez durante a execução. Em aplicações modernas, porém, frameworks de injeção de dependência frequentemente tornam o uso manual de Singleton desnecessário ou problemático.

Cuidado de prova e prática. Singleton não significa classe estática. Também não significa que o objeto é imutável, thread-safe ou adequado para todos os cenários. Em ambiente concorrente, a implementação precisa cuidar de sincronização, inicialização preguiçosa e visibilidade de memória.

3.2 Factory Method

Factory Method **define uma interface ou método para criação de objetos**, mas permite que **subclasses decidam qual classe concreta será instanciada**. A intenção central é adiar a escolha da classe concreta para subclasses ou implementações específicas.

Exemplo: uma classe abstrata `ProcessadorDocumento` define o método `criarValidador()`. Subclasses como `ProcessadorPDF` e `ProcessadorXML` retornam validadores específicos. O cliente trabalha com a abstração.

Diferença crítica. Factory Method é sobre um método de criação, normalmente em uma hierarquia. Abstract Factory é sobre uma fábrica de famílias de objetos relacionados.

3.3 Abstract Factory

Abstract Factory fornece uma **interface para criar famílias de objetos relacionados ou dependentes, sem especificar suas classes concretas**. A FGV costuma cobrar a expressão “famílias de objetos”.

Exemplo: uma aplicação multi-tribunal precisa gerar componentes visuais e validadores de protocolo de acordo com o tribunal. Uma `FabricaTJSC` cria `CabecalhoTJSC`, `ValidadorTJSC` e `AssinadorTJSC`; uma `FabricaTJRJ` cria a família equivalente para outro tribunal.

3.4 Builder

Builder separa a **construção de um objeto complexo de sua representação final, permitindo criar diferentes representações pelo mesmo processo**. É típico quando o objeto possui muitos passos, **muitos parâmetros opcionais ou montagem incremental**.

Exemplo prático: construir um `RelatorioProcessual` com cabeçalho, partes, movimentos, anexos e metadados. Em Java, builders também reduzem construtores telescópicos e deixam a criação mais legível.

Pegadinha. Builder não é “qualquer método que cria objeto”. Se a questão enfatiza montagem passo a passo de objeto complexo, pense em Builder; se enfatiza escolha de subclasse concreta, pense em Factory Method.

3.5 Prototype

Prototype cria novos objetos copiando uma instância protótipo existente. O foco é clonagem. Aparece quando criar do zero é caro ou quando se deseja parametrizar objetos a partir de modelos previamente configurados.

Exemplo: um modelo de documento processual com metadados padrão é clonado para gerar novas minutas, ajustando apenas partes específicas.

4. Padrões estruturais

Padrões estruturais organizam classes e objetos em estruturas maiores. São muito cobrados por alternativas próximas: Adapter, Facade, Decorator e Proxy frequentemente aparecem no mesmo item.

Padrão	Essência	Como a FGV tenta confundir
Adapter	Converte a interface de uma classe em outra esperada pelo cliente.	Confundir com Facade, porque ambos “simplificam” uso. Adapter resolve incompatibilidade de interface.
Facade	Oferece uma interface simplificada para um subsistema complexo.	Confundir com Adapter. Facade não precisa converter interface incompatível; ela encapsula complexidade.
Decorator	Adiciona responsabilidades dinamicamente a um objeto, preservando interface.	Confundir com herança. Decorator favorece composição e extensão em tempo de execução.
Proxy	Representa/substitui outro objeto para controlar acesso, criação, cache ou comunicação remota.	Confundir com Decorator. Proxy controla acesso; Decorator adiciona comportamento.

4.1 Adapter

Adapter permite que classes com interfaces incompatíveis trabalhem juntas. Ele traduz chamadas de uma interface esperada para outra interface existente.

Exemplo: o sistema interno espera a interface `ServicoPagamento`, mas um provedor legado expõe `executarLiquidacao()` com outro contrato. Um adapter encapsula o legado e oferece a interface esperada pelo módulo novo.

4.2 Facade

Facade fornece uma interface unificada e simplificada para um conjunto de classes ou subsistemas. Em APIs corporativas, aparece como serviço de aplicação que orquestra validação, persistência, auditoria e mensageria sem expor cada detalhe ao cliente.

Cuidado. Facade não elimina a complexidade interna; apenas a esconde por trás de uma interface mais simples.

4.3 Decorator

Decorator adiciona responsabilidades a objetos dinamicamente, compondo objetos em camadas. O objeto decorador implementa a mesma interface do objeto decorado e delega chamadas, acrescentando comportamento.

Exemplo: um serviço de envio de notificação pode receber decoradores para logging, métricas, auditoria ou validação de autorização, sem alterar a classe original.

4.4 Proxy

Proxy fornece um substituto ou representante de outro objeto para controlar acesso a ele. Pode ser proxy remoto, virtual, de proteção ou de cache.

Exemplo: um proxy para um serviço externo de consulta processual decide quando autenticar, quando cachear resposta, quando bloquear acesso por perfil ou quando criar a conexão real.

4.5 Composite, Bridge e Flyweight

Composite **compõe objetos em estruturas de árvore para tratar objetos individuais e composições de forma uniforme**. Exemplo: menus, árvores de unidades, hierarquias de documentos ou componentes de interface.

Bridge **separa abstração de implementação para que ambas possam variar independentemente**. É menos cobrado que Adapter/Facade, mas aparece em questões de distinção fina.

Flyweight **compartilha objetos de granularidade fina para economizar memória, separando estado intrínseco compartilhável de estado extrínseco fornecido pelo cliente**.

5. Padrões comportamentais

Padrões comportamentais tratam da comunicação entre objetos, variação de algoritmos, estados, comandos e responsabilidades. A FGV costuma cobrar Strategy, Observer, State, Command e Template Method.

5.1 Strategy

Strategy define uma família de algoritmos, encapsula cada um deles e os torna intercambiáveis. O algoritmo varia independentemente do cliente que o utiliza. Palavra-chave: algoritmo selecionável/intercambiável em tempo de execução.

Exemplo: cálculo de prioridade de atendimento pode variar por regra: antiguidade, urgência, classe processual ou perfil de usuário. O serviço usa uma interface EstrategiaPriorizacao e escolhe a implementação adequada.

5.2 Observer

Observer define uma dependência um-para-muitos entre objetos: quando o sujeito muda de estado, seus observadores são notificados automaticamente.

Exemplo: ao protocolar um documento, módulos de notificação, auditoria e atualização de painel são avisados. Em arquitetura moderna, eventos e mensageria podem materializar ideia semelhante, mas a questão de padrão de projeto costuma focar na relação sujeito-observadores.

5.3 State

State permite que um objeto altere seu comportamento quando seu estado interno muda, parecendo mudar de classe. É comum em fluxos com estados fortes: Rascunho, Protocolado, Em análise, Arquivado.

Diferença para Strategy. Strategy troca algoritmo por escolha de política; State muda comportamento conforme estado interno do próprio objeto. Ambos encapsulam comportamento, mas a motivação é diferente.

5.4 Command

Command encapsula uma solicitação como objeto, permitindo parametrizar ações, enfileirar operações, registrar histórico, desfazer/refazer ou executar posteriormente.

Exemplo: comandos de “assinar documento”, “juntar peça”, “arquivar processo” e “reabrir prazo” podem ser representados como objetos com execução, auditoria e eventual compensação.

5.5 Template Method

Template Method define o esqueleto de um algoritmo em uma operação, deixando alguns passos para subclasses. A estrutura geral fica fixa; partes variáveis são especializadas.

Exemplo: um processo de importação tem passos fixos: ler arquivo, validar, transformar, persistir e emitir relatório. Cada formato implementa passos específicos, mas o fluxo geral é preservado.

5.6 Outros comportamentais cobrados por distinção

Padrão	Ideia de prova
Iterator	Percorrer elementos de uma coleção sem expor sua representação interna.
Mediator	Centralizar a comunicação entre objetos para reduzir dependências diretas muitos-para-muitos.
Chain of Responsibility	Passar uma requisição por uma cadeia de handlers até algum tratá-la.
Visitor	Adicionar operações a uma estrutura de objetos sem alterar as classes dos elementos.
Memento	Capturar e restaurar estado interno sem violar encapsulamento.

Padrão	Ideia de prova
Interpreter	Representar gramática e interpretar sentenças de uma linguagem.

6. Comparações que mais derrubam

Confusão	Distinção para acertar
Factory Method x Abstract Factory	Factory Method cria um produto por método especializado; Abstract Factory cria famílias de produtos relacionados.
Builder x Factory	Builder monta objeto complexo passo a passo; Factory decide qual classe concreta instanciar.
Prototype x Factory	Prototype cria por clonagem de instância existente; Factory cria por lógica de criação.
Adapter x Facade	Adapter compatibiliza interfaces; Facade simplifica acesso a subsistema complexo.
Decorator x Proxy	Decorator adiciona responsabilidades; Proxy controla acesso, criação, cache, proteção ou comunicação remota.
Strategy x State	Strategy varia algoritmo por escolha; State varia comportamento conforme estado interno.
Observer x Mediator	Observer notifica interessados sobre mudança de estado; Mediator coordena comunicação entre objetos para reduzir acoplamento.
Template Method x Strategy	Template Method usa herança e fixa esqueleto do algoritmo; Strategy usa composição e troca algoritmo.

6.1 Catálogo compacto dos padrões GoF para prova

A FGV não costuma exigir que você desenhe a estrutura formal de todos os 23 padrões GoF, mas pode cobrar nomes, categorias e intenções. O quadro abaixo funciona como mapa de eliminação. Em prova, identifique primeiro a família: criação, estrutura ou comportamento. Depois procure a palavra-chave do problema.

Família	Padrão	Intenção enxuta para prova
Criacional	Factory Method	Subclasses/implementações decidem qual produto concreto criar.
Criacional	Abstract Factory	Cria famílias de objetos relacionados sem acoplar a classes concretas.
Criacional	Builder	Constrói objeto complexo passo a passo, separando construção e representação.
Criacional	Prototype	Cria objetos por cópia/clonagem de protótipo.
Criacional	Singleton	Garante uma única instância com ponto controlado de acesso.
Estrutural	Adapter	Converte interface incompatível em interface esperada pelo cliente.
Estrutural	Bridge	Separa abstração de implementação para variar independentemente.
Estrutural	Composite	Trata objetos individuais e composições em árvore de forma uniforme.
Estrutural	Decorator	Adiciona responsabilidades dinamicamente por composição.
Estrutural	Facade	Fornece interface simplificada para subsistema complexo.
Estrutural	Flyweight	Compartilha objetos pequenos e numerosos para economizar memória.
Estrutural	Proxy	Controla acesso a outro objeto ou o representa.
Comportamental	Chain of Responsibility	Passa requisição por cadeia de tratadores.
Comportamental	Command	Encapsula solicitação como objeto.
Comportamental	Interpreter	Interpreta sentenças de uma linguagem/gramática.
Comportamental	Iterator	Percorre coleção sem expor representação interna.
Comportamental	Mediator	Centraliza comunicação entre objetos para reduzir acoplamento.
Comportamental	Memento	Captura/restaura estado sem violar encapsulamento.

Família	Padrão	Intenção enxuta para prova
Comportamental	Observer	Notifica observadores quando sujeito muda de estado.
Comportamental	State	Varia comportamento conforme estado interno.
Comportamental	Strategy	Encapsula algoritmos intercambiáveis.
Comportamental	Template Method	Fixa esqueleto de algoritmo e delega passos a subclasses.
Comportamental	Visitor	Adiciona operações a estrutura de objetos sem alterar elementos.

6.2 Exemplos de pseudocódigo que ajudam a reconhecer padrões

A prova pode apresentar trechos de código ou pseudocódigo. Você não precisa implementar padrões do zero durante a prova, mas deve reconhecer a intenção.

Strategy em pseudocódigo:

```
interface PoliticaPrioridade { int calcular(Processo p); }
class PrioridadePorUrgencia implements PoliticaPrioridade { ... }
class PrioridadePorAntiguidade implements PoliticaPrioridade { ... }
class ServicoFila { PoliticaPrioridade politica; int calcular(Processo p) { return politica.calcular(p); } }
```

O sinal de Strategy é a troca do algoritmo por composição. Se o enunciado disser que a política é escolhida em tempo de execução, a pista fica ainda mais forte.

Template Method em pseudocódigo:

```
abstract class Importador { final void importar() { abrir(); validar(); transformar(); salvar(); } abstract void transformar(); }
```

O sinal de Template Method é o método que fixa a sequência geral e deixa passos para subclasses. Se o enunciado falar em “esqueleto do algoritmo”, quase sempre é Template Method.

Adapter em pseudocódigo:

```
class AdaptadorServicoLegado implements ServicoNovo { Legado legado; Resposta consultar(Requisicao r) { return converter(legado.executar(formatoAntigo(r))); } }
```

O sinal de Adapter é a tradução entre contratos. Não é apenas simplificar uso; é compatibilizar interface esperada e interface existente.

6.3 Padrões, SOLID e acoplamento

Padrões de projeto costumam aparecer junto de princípios de bom projeto, mas a FGV pode explorar a diferença: **SOLID é conjunto de princípios; padrões são soluções recorrentes**. Strategy, por exemplo, ajuda a aplicar OCP - Open/Closed Principle, pois novas políticas podem ser adicionadas sem alterar o cliente. Adapter ajuda a reduzir acoplamento com um legado. Facade reduz dependência dos consumidores em relação a múltiplas classes internas.

Padrão	Princípio/efeito associado	Armadilha
Strategy	Favorece extensão por novas estratégias e reduz condicionais.	Não confundir com State: aqui a variação é algoritmo/política.
Factory Method	Reduz dependência do cliente em classes concretas.	Não confundir com Abstract Factory quando o problema é família de produtos.
Adapter	Isola acoplamento com contrato legado ou externo.	Não é obrigatoriamente uma Facade.
Decorator	Favorece composição em vez de herança para adicionar comportamento.	Não é simplesmente usar annotation/decorator de linguagem.
Facade	Reduz acoplamento do cliente com subsistemas internos.	Pode virar “classe Deus” se concentrar regra demais.

6.4 Armadilhas de nomenclatura que a FGV pode usar

Termo no enunciado	Padrão mais provável	Atenção
Única instância, ponto global/controlado	Singleton	Não garante thread safety por si só.
Família de objetos relacionados	Abstract Factory	Não é apenas criar um objeto.
Subclasse decide produto concreto	Factory Method	Não confundir com Builder.
Objeto complexo, passos de construção	Builder	Muitos parâmetros opcionais também são pista.
Clonar, copiar objeto protótipo	Prototype	Foco é cópia, não fábrica.
Interface incompatível	Adapter	Tradução de contrato.
Interface simples para subsistema	Facade	Ocultar complexidade.
Responsabilidades dinâmicas	Decorator	Composição em camadas.
Controle de acesso, cache, remoto, proteção	Proxy	Representa outro objeto.
Notificação de dependentes	Observer	Sujeito muda; observadores são avisados.
Algoritmos intercambiáveis	Strategy	Escolha de política.
Estado interno altera comportamento	State	Fluxo por estados.
Requisição como objeto	Command	Fila, log, undo/redo.
Cadeia de validadores	Chain of Responsibility	Um handler trata ou repassa.

7. Ferramentas CASE - Computer-Aided Software Engineering

CASE - Computer-Aided Software Engineering **designa ferramentas que automatizam, apoiam ou disciplinam atividades de engenharia de software**. A ideia central é suporte ao processo: análise, modelagem, projeto, geração de documentação, geração de código, engenharia reversa, rastreabilidade, verificação de consistência, controle de versões de modelos e integração entre artefatos.

A FGV pode cobrar CASE de modo simples: “ferramentas CASE auxiliam o desenvolvimento e manutenção de software”. O erro comum é reduzir CASE a uma IDE - Integrated Development Environment, ou confundir com ferramenta de desenho sem semântica de engenharia.

7.1 Upper CASE, Lower CASE e I-CASE

Categoria	Foco	Exemplos de atividades	Pegadinha
Upper CASE	Fases iniciais: negócio, requisitos, análise e projeto.	Modelagem de processos, DER, DFD, UML, casos de uso, dicionário de dados, especificação.	Não é ferramenta “superior” por qualidade; é superior no ciclo de vida.
Lower CASE	Fases posteriores: implementação, testes, manutenção e suporte técnico.	Geração de código, depuração, teste, build, manutenção, engenharia reversa.	Não significa ferramenta menos importante.
I-CASE ou Integrated CASE	Integração ao longo do ciclo de vida com repositório comum.	Rastreabilidade de requisitos a modelos, código, testes e documentação.	Não basta ter várias ferramentas isoladas; integração e repositório fazem diferença.

7.2 Recursos típicos de CASE

Recursos típicos de ferramentas CASE incluem: modelagem UML - Unified Modeling Language; modelagem de dados; geração de código a partir de modelos; engenharia reversa de código para diagramas; dicionário ou repositório de metadados; checagem de consistência; documentação automática; rastreabilidade; controle de versões de artefatos; apoio à gestão de requisitos.

Em sistemas corporativos, uma ferramenta de modelagem UML com geração de classes Java, uma ferramenta de modelagem de banco que gera DDL - Data Definition Language, ou uma plataforma ALM - Application Lifecycle Management com rastreabilidade entre requisito, modelo, issue, commit e teste pode exercer funções CASE.

7.3 CASE, UML e MDA

Ferramentas CASE frequentemente apoiam UML, mas CASE não é sinônimo de UML. UML é uma linguagem de modelagem; CASE é a ferramenta ou ambiente que apoia atividades de engenharia.

MDA - Model Driven Architecture, da OMG - Object Management Group, é uma **abordagem de projeto, desenvolvimento e implementação baseada em modelos**. Em prova, MDA/MDD pode aparecer como contexto de geração de código e transformação de modelos, mas a aula limita esse ponto ao que ajuda a entender CASE: modelos com semântica, transformação e rastreabilidade podem ser usados como artefatos centrais de desenvolvimento.

7.5 Engenharia direta, engenharia reversa e round-trip engineering

Engenharia direta é **gerar artefatos de implementação a partir de modelos**: por exemplo, gerar classes, interfaces, scripts DDL - Data Definition Language ou documentação a partir de diagramas. Engenharia reversa é reconstruir modelos a partir de código, banco ou artefatos existentes. Round-trip engineering tenta manter sincronismo entre modelo e código nos dois sentidos.

A FGV pode transformar esses conceitos em caso prático: se o enunciado diz que a ferramenta lê código Java e gera diagrama de classes, pense em engenharia reversa; se diz que lê diagrama e gera código ou DDL, pense em engenharia direta.

Situação	Nome provável
Modelo UML gera classes Java e interfaces TypeScript.	Engenharia direta
Banco existente gera DER ou modelo relacional visual.	Engenharia reversa
Código e modelo são sincronizados após alterações em ambos.	Round-trip engineering
Ferramenta apenas desenha caixas e setas sem semântica.	Não é CASE forte; é desenho/diagramação

7.6 CASE em cada fase do ciclo de vida

Fase/atividade	Apoio CASE típico	Exemplo no seu contexto
Requisitos	Catálogo de requisitos, rastreabilidade, matriz requisito-teste.	Regra de negócio de autorização vinculada a caso de teste e issue.
Análise	Casos de uso, modelo de domínio, fluxos, BPMN, glossário.	Mapear ator, API interna, sistema legado e fluxo de aprovação.
Projeto	Diagramas de classes, componentes, sequência, implantação, padrões.	Desenhar serviço Quarkus, adaptador de legado e facade de API.
Implementação	Geração de código, esqueletos de classes, templates, validação de estrutura.	Gerar DTOs, interfaces e stubs a partir de modelo.
Teste	Rastreabilidade requisito-teste, geração de casos, cobertura de regras.	Vincular requisito de retenção a teste de integração.
Manutenção	Engenharia reversa, análise de impacto, documentação automática.	Reconstruir dependências de módulo legado para refatoração.

7.4 Diferença entre CASE, IDE, ferramenta de desenho e DevOps

Ferramenta	Papel típico	Como não confundir
CASE	Apoia atividades de engenharia de software, especialmente modelagem, análise, projeto, documentação, geração e rastreabilidade.	Tem foco em artefatos de engenharia e ciclo de vida.
IDE	Ambiente de edição, compilação, depuração e execução de código.	Pode ter funções CASE, mas IDE pura não é sinônimo de CASE.
Ferramenta de desenho	Desenha figuras sem necessariamente manter semântica de modelo.	Se não valida modelo nem gera artefatos, é apenas desenho.
Ferramenta DevOps	Automatiza build, testes, deploy, observabilidade e operação.	Pode integrar ciclo de vida, mas não é automaticamente CASE se não apoia modelagem/engenharia.

8. Exemplos conectados ao seu contexto

Java/Quarkus. Em uma API de integrações, Strategy pode escolher a política de validação por tipo de pagamento; Factory Method pode criar clientes para provedores diferentes; Adapter pode encapsular um serviço legado com contrato incompatível; Facade pode oferecer uma operação única de “autorizar pagamento” que orquestra validação, persistência e auditoria.

Angular/TypeScript. Observer aparece em fluxos reativos e atualização de componentes; Decorator aparece no ecossistema Angular com anotações e metadados, mas cuidado: o decorator da linguagem/framework não é automaticamente o padrão GoF Decorator. A prova normalmente cobra a intenção de adicionar responsabilidades dinamicamente mantendo interface.

Sistemas públicos e documentação. CASE pode apoiar modelagem de processos, requisitos, UML, rastreabilidade de regras, geração de documentação técnica e integração com repositórios. Em um tribunal, o valor é reduzir ambiguidade, preservar histórico de decisões de projeto e manter coerência entre requisito, modelo, implementação e teste.

9. Lista de questões autorais - padrão FGV

Resolva as 20 questões antes de consultar o gabarito. As questões são inéditas, com formatos variados e distratores próximos, calibradas por padrões observados em provas FGV de TI.

1. Em um sistema corporativo de tramitação de documentos, a equipe decidiu que a criação de validadores deve depender do tipo de documento, mas o módulo cliente não deve conhecer as classes concretas ValidadorPDF, ValidadorXML ou ValidadorHTML. A classe base define uma operação de criação, e subclasses especializadas decidem qual validador concreto instanciar. O padrão de projeto descrito é:

- (A) Abstract Factory.
- (B) Builder.
- (C) Factory Method.
- (D) Decorator.
- (E) Strategy.

2. Em uma arquitetura de geração de interfaces, uma aplicação precisa criar famílias compatíveis de objetos: botões, validadores e componentes de layout para cada tribunal integrado. A escolha de uma família deve impedir combinações inconsistentes entre componentes de tribunais diferentes. O padrão mais adequado é:

- (A) Abstract Factory.
- (B) Prototype.
- (C) Singleton.
- (D) Facade.
- (E) Iterator.

3. Considere as afirmativas sobre padrões de projeto. I. Padrões de projeto são soluções recorrentes, nomeadas e reutilizáveis para problemas de projeto, mas não são implementações prontas. II. A classificação GoF divide padrões em criacionais, estruturais e comportamentais. III. Um padrão de projeto sempre define a arquitetura completa da aplicação, incluindo implantação, rede e infraestrutura. Está correto o que se afirma em:

- (A) I, apenas.
- (B) II, apenas.
- (C) I e III, apenas.
- (D) II e III, apenas.
- (E) I e II, apenas.

4. Um módulo de consulta processual precisa usar um serviço legado cujo método principal recebe parâmetros e retorna dados em formato incompatível com a interface esperada pelo novo domínio. A equipe cria uma classe intermediária que implementa a interface esperada e traduz as chamadas para o contrato legado. Esse caso caracteriza o padrão:

- (A) Facade.
- (B) Adapter.
- (C) Observer.
- (D) Template Method.
- (E) Composite.

5. Em uma API Java, a operação `protocolarDocumento` executa internamente validação, autorização, persistência, registro de auditoria e publicação de evento. Para consumidores externos, a equipe expõe uma interface simples, escondendo a complexidade do subsistema. O padrão de projeto mais aderente é:

- (A) Adapter.
- (B) Proxy.
- (C) Decorator.
- (D) Facade.
- (E) Prototype.

6. A respeito de padrões estruturais, assinale a opção que apresenta corretamente a distinção entre Decorator e Proxy.

- (A) Decorator controla acesso remoto, enquanto Proxy monta objetos complexos passo a passo.
- (B) Decorator cria famílias de objetos relacionados, enquanto Proxy percorre coleções.
- (C) Decorator adiciona responsabilidades dinamicamente ao objeto, enquanto Proxy controla o acesso ou representa outro objeto.
- (D) Decorator centraliza comunicação entre objetos, enquanto Proxy encapsula algoritmos intercambiáveis.
- (E) Decorator garante instância única, enquanto Proxy define o esqueleto de algoritmo em subclasse.

7. Relacione os padrões às descrições. 1. Observer. 2. Strategy. 3. Singleton. 4. Builder. 5. Command. () Encapsula uma requisição como objeto. () Garante uma única instância com ponto de acesso controlado. () Encapsula famílias de algoritmos intercambiáveis. () Notifica dependentes quando o sujeito muda. () Constrói objeto complexo passo a passo. A sequência correta é:

- (A) 5 - 3 - 2 - 1 - 4.
- (B) 3 - 5 - 2 - 1 - 4.
- (C) 5 - 2 - 3 - 4 - 1.
- (D) 4 - 3 - 1 - 2 - 5.
- (E) 1 - 3 - 2 - 5 - 4.

8. Uma classe ImportadorBase define o fluxo fixo para importar dados: abrir arquivo, validar cabeçalho, converter registros, persistir dados e gerar relatório. Subclasses sobrescrevem apenas etapas específicas, sem alterar a ordem geral do processo. O padrão descrito é:

- (A) State.
- (B) Strategy.
- (C) Visitor.
- (D) Chain of Responsibility.
- (E) Template Method.

9. Em um fluxo de processo eletrônico, o comportamento do objeto Processo muda conforme seu estado: em rascunho ele aceita edição; protocolado não aceita exclusão; arquivado aceita apenas consulta e reabertura controlada. Para evitar uma sequência crescente de condicionais, a equipe encapsula esses comportamentos em classes de estado. O padrão empregado é:

- (A) Strategy.
- (B) State.
- (C) Observer.
- (D) Prototype.
- (E) Flyweight.

10. Assinale a opção incorreta sobre ferramentas CASE - Computer-Aided Software Engineering.

- (A) Podem apoiar modelagem, documentação, rastreabilidade, geração de código e engenharia reversa.
- (B) Podem usar repositório de metadados para integrar artefatos de requisitos, análise, projeto e implementação.
- (C) Substituem integralmente a análise humana, eliminando a necessidade de validação com usuários e stakeholders.
- (D) Podem ser classificadas, em uma visão clássica, como Upper CASE, Lower CASE e Integrated CASE.
- (E) Podem apoiar atividades de UML, modelagem de dados, dicionário de dados e consistência de modelos.

11. Uma ferramenta permite modelar processos de negócio, diagramas de casos de uso, diagramas de classes, entidades de dados e requisitos, mantendo rastreabilidade entre os artefatos. Segundo a classificação clássica de CASE, o foco principal descrito é:

- (A) Lower CASE, pois automatiza exclusivamente compilação e deploy.
- (B) Ferramenta de monitoramento, pois coleta métricas de produção.
- (C) Biblioteca de padrões, pois substitui o projeto por componentes prontos.
- (D) Upper CASE, pois apoia atividades de análise, requisitos e projeto.
- (E) Proxy CASE, pois controla o acesso remoto a modelos.

12. Considere as afirmativas sobre CASE. I. Upper CASE apoia atividades mais próximas de análise e projeto. II. Lower CASE apoia atividades mais próximas de implementação, testes e manutenção. III. I-CASE pressupõe integração de ferramentas e, em geral, repositório comum ou rastreabilidade entre artefatos. Está correto o que se afirma em:

- (A) I, apenas.
- (B) II, apenas.
- (C) I e II, apenas.
- (D) II e III, apenas.
- (E) I, II e III.

13. Um time Angular precisa reagir automaticamente à mudança de estado de um serviço de autenticação, atualizando componentes interessados sem acoplá-los diretamente à lógica interna do serviço. Em termos de padrão de projeto, a intenção mais próxima é:

- (A) Observer.
- (B) Builder.
- (C) Abstract Factory.
- (D) Prototype.
- (E) Facade.

14. Em uma aplicação, várias políticas de cálculo de prioridade podem ser selecionadas em tempo de execução: prioridade por urgência, por data de entrada, por classe processual ou por perfil de usuário. O cliente usa uma interface comum e troca a implementação conforme a regra escolhida. Esse projeto corresponde ao padrão:

- (A) State.
- (B) Adapter.
- (C) Strategy.
- (D) Proxy.
- (E) Composite.

15. Observe as afirmações: I. Adapter e Facade são padrões estruturais. II. Adapter visa compatibilizar interfaces incompatíveis, enquanto Facade visa simplificar o acesso a subsistema complexo. III. Facade é obrigatoriamente implementado por herança múltipla, pois precisa expor todas as interfaces internas. Está correto o que se afirma em:

- (A) I, apenas.
- (B) I e II, apenas.
- (C) II e III, apenas.
- (D) I e III, apenas.
- (E) I, II e III.

16. Um sistema precisa representar uma hierarquia de componentes visuais em árvore, permitindo que o cliente trate de modo uniforme um botão simples e um painel composto por vários componentes filhos. O padrão de projeto aplicável é:

- (A) Bridge.
- (B) Flyweight.
- (C) Decorator.
- (D) Composite.
- (E) Singleton.

17. Uma equipe usa uma ferramenta que, a partir de código Java existente, reconstrói diagramas de classes e dependências para apoiar documentação e manutenção. Esse recurso é conhecido, no contexto de CASE, como:

- (A) engenharia reversa.
- (B) compilação incremental.
- (C) serialização de objetos.
- (D) implantação contínua.
- (E) normalização relacional.

18. Uma ferramenta transforma um modelo de classes em esqueletos de código Java e scripts de criação de tabelas, mantendo vínculos entre elementos do modelo e artefatos gerados. O caso se aproxima mais de:

- (A) uma ferramenta meramente gráfica sem semântica de engenharia.
- (B) uma biblioteca de execução de algoritmos de ordenação.

- (C) um padrão comportamental do tipo Iterator.
- (D) um proxy de acesso a banco de dados.
- (E) uma ferramenta CASE com geração direta de artefatos a partir de modelos.

19. Sobre padrões de projeto e padrões arquiteturais, assinale a opção correta.

- (A) Padrões de projeto GoF sempre descrevem topologia de implantação e comunicação entre servidores.
- (B) MVC e DDD pertencem necessariamente à mesma categoria GoF dos padrões criacionais.
- (C) Padrões de projeto atuam tipicamente em nível de classes/objetos e responsabilidades, enquanto padrões arquiteturais tratam da organização macro do sistema.
- (D) Singleton e Observer são padrões arquiteturais de integração entre sistemas distribuídos.
- (E) Ferramentas CASE são padrões estruturais usados para adaptar interfaces incompatíveis.

20. Um sistema de workflow passa uma solicitação por uma sequência de validadores: autorização do usuário, integridade dos dados, prazo processual, limite de tamanho e política de sigilo. Cada validador pode tratar a requisição ou repassá-la ao próximo. O padrão mais adequado é:

- (A) Template Method.
- (B) Chain of Responsibility.
- (C) Abstract Factory.
- (D) Memento.
- (E) Builder.

10. Gabarito resumido

1-C, 2-A, 3-E, 4-B, 5-D, 6-C, 7-A, 8-E, 9-B, 10-C, 11-D, 12-E, 13-A, 14-C, 15-B, 16-D, 17-A, 18-E, 19-C, 20-B.

11. Comentários completos

Questão 1-C

Factory Method define uma operação de criação e permite que subclasses decidam a classe concreta. A alternativa A seduz porque Abstract Factory também cria objetos, mas seu foco é família de produtos relacionados. B é montagem passo a passo de objeto complexo. D adiciona responsabilidades dinamicamente. E encapsula algoritmos intercambiáveis, não criação de validadores por subclasse.

Questão 2-A

Abstract Factory é o padrão de famílias de objetos relacionados. A palavra-chave do enunciado é “famílias compatíveis”. B, Prototype, criaria por clonagem. C, Singleton, controlaria instância única. D, Facade, simplificaria subsistema. E, Iterator, percorreria coleções.

Questão 3-E

I e II estão corretas: padrões são soluções recorrentes, não código pronto; e a classificação GoF é criacional, estrutural e comportamental. III está errada porque padrão de projeto não define necessariamente arquitetura completa, implantação, rede e infraestrutura. Essa é uma confusão típica entre padrão de projeto e padrão arquitetural.

Questão 4-B

Adapter compatibiliza interfaces incompatíveis por meio de uma classe intermediária que traduz chamadas. A, Facade, simplifica subsistema, mas não necessariamente adapta interface. C, Observer, notifica dependentes. D, Template Method, fixa esqueleto de algoritmo. E, Composite, estrutura objetos em árvore.

Questão 5-D

Facade oferece interface simples para um subsistema complexo. A operação única que esconde validação, autorização, persistência, auditoria e evento é típica de Facade. Adapter exigiria incompatibilidade de interfaces. Proxy controlaria acesso. Decorator acrescentaria responsabilidade mantendo interface. Prototype clonaria objetos.

Questão 6-C

Decorator adiciona responsabilidades dinamicamente ao objeto, preservando interface; Proxy representa outro objeto para controlar acesso, criação, cache, proteção ou comunicação remota. A troca Proxy com Builder. B mistura Abstract Factory e Iterator. D mistura Mediator e Strategy. E mistura Singleton e Template Method.

Questão 7-A

Command encapsula solicitação como objeto; Singleton garante instância única; Strategy encapsula algoritmos intercambiáveis; Observer notifica dependentes; Builder constrói objeto complexo passo a passo. As demais sequências invertem padrões próximos, especialmente Observer/Strategy e Builder/Command.

Questão 8-E

Template Method define o esqueleto de algoritmo e deixa etapas para subclasses. A, State, depende de estado interno. B, Strategy, troca algoritmo por composição. C, Visitor, adiciona operações a estrutura de objetos. D, Chain of Responsibility, passa requisição por cadeia de handlers.

Questão 9-B

State encapsula comportamentos que variam conforme estado interno do objeto. A, Strategy, também encapsula comportamento, mas a variação decorre de política selecionada, não do estado interno do próprio objeto. C, Observer, notifica. D, Prototype, clona. E, Flyweight, compartilha objetos para economizar memória.

Questão 10-C

A incorreta é C. CASE apoia e automatiza atividades, mas não substitui integralmente análise humana nem validação com usuários. A, B, D e E descrevem recursos ou classificações clássicas de CASE: modelagem,

documentação, rastreabilidade, geração, engenharia reversa, repositório, Upper/Lower/I-CASE e apoio a UML/modelagem.

Questão 11-D

Upper CASE apoia fases iniciais: análise, requisitos, modelagem e projeto. A descreve Lower CASE, mas o enunciado não fala de compilação/deploy. B é monitoramento. C reduz a questão a biblioteca de padrões. E inventa uma categoria inexistente, usando “proxy” fora do contexto.

Questão 12-E

As três afirmativas estão corretas. Upper CASE fica mais próximo de requisitos/análise/projeto; Lower CASE, de implementação/testes/manutenção; I-CASE envolve integração e rastreabilidade entre ferramentas/artefatos. A banca pode tentar trocar “upper” por qualidade superior, mas a classificação é por fase do ciclo de vida.

Questão 13-A

Observer é a intenção de notificar interessados automaticamente quando há mudança de estado. B é criação complexa. C é criação de famílias. D é clonagem. E simplifica subsistema. Em Angular, fluxos reativos podem lembrar Observer, mas a prova cobra a intenção conceitual, não a biblioteca específica.

Questão 14-C

Strategy encapsula uma família de algoritmos e permite selecioná-los em tempo de execução. A, State, variaria por estado interno. B, Adapter, compatibilizaria interfaces. D, Proxy, controlaria acesso. E, Composite, trataria árvore de objetos de modo uniforme.

Questão 15-B

I e II estão corretas. Adapter e Facade são estruturais; Adapter compatibiliza interfaces, Facade simplifica subsistema. III está errada: Facade não exige herança múltipla nem expõe todas as interfaces internas. Pelo contrário, a intenção é reduzir a exposição de complexidade.

Questão 16-D

Composite compõe objetos em árvore e permite tratar objeto simples e composição de maneira uniforme. A, Bridge, separa abstração e implementação. B, Flyweight, compartilha objetos finos. C, Decorator, adiciona responsabilidades. E, Singleton, controla instância única.

Questão 17-A

Reconstruir modelos a partir de código existente é engenharia reversa no contexto CASE. B é técnica de build. C é persistência/representação de objetos. D pertence a CI/CD. E é conceito de banco de dados. A questão explora confusão entre manutenção de software e atividades de banco/DevOps.

Questão 18-E

Geração de código e scripts a partir de modelos, com vínculo entre artefatos, é recurso típico de ferramenta CASE. A está errada porque o enunciado indica semântica e geração, não mero desenho. B, C e D deslocam o assunto para algoritmos, Iterator e Proxy.

Questão 19-C

Padrões de projeto atuam normalmente no nível de classes/objetos, responsabilidades e colaboração; padrões arquiteturais organizam o sistema em nível macro. A exagera o alcance dos GoF. B classifica MVC/DDD indevidamente como categoria GoF criacional. D transforma Singleton/Observer em padrões de integração distribuída. E confunde CASE com Adapter.

Questão 20-B

Chain of Responsibility passa uma requisição por uma cadeia de objetos até que algum trate ou todos processem. A, Template Method, fixaria esqueleto de algoritmo. C criaria famílias. D capturaria estado para restauração. E montaria objeto complexo passo a passo.

12. Checklist final do que memorizar

- GoF = Gang of Four; categorias: criacionais, estruturais e comportamentais.
- Factory Method = subclasses decidem classe concreta; Abstract Factory = famílias de objetos relacionados.
- Builder = montagem passo a passo de objeto complexo; Prototype = clonagem; Singleton = instância única.
- Adapter = compatibiliza interfaces; Facade = simplifica subsistema; Decorator = adiciona responsabilidade; Proxy = controla acesso/representa objeto.
- Strategy = algoritmo intercambiável; State = comportamento varia conforme estado interno; Observer = notificação automática; Command = requisição como objeto; Template Method = esqueleto fixo com passos em subclasses.
- CASE = Computer-Aided Software Engineering: apoio automatizado à engenharia de software.
- Upper CASE = análise/projeto; Lower CASE = implementação/testes/manutenção; I-CASE = integração e repositório/rastreabilidade.
- CASE não é sinônimo de IDE, ferramenta de desenho, UML ou DevOps, embora possa se integrar a esses elementos.

13. Referências consultadas

- TJSC - Edital n. 10/2026, retificado: disciplina Engenharia de Software e Desenvolvimento, itens 1.9 Padrões de projeto e 1.11 Ferramentas CASE.
- Manual Mestre de Calibração FGV para Aulas e Questões - Projeto TJSC 2026.
- FGV Conhecimento - provas oficiais de TI consultadas para calibrar formato: Analista em Geociências - Análise e Desenvolvimento de Sistemas; Analista Legislativo - Analista de Sistemas; Analista Legislativo - Análise de Sistemas.
- Gamma, Helm, Johnson e Vlissides - Design Patterns: Elements of Reusable Object-Oriented Software - referência conceitual clássica dos padrões GoF.
- OMG - Object Management Group: Model Driven Architecture - referência sobre abordagem dirigida a modelos.
- Oracle: Overview to Computer Aided Software Engineering - referência sobre CASE, Upper CASE e Lower CASE.
- Refactoring.Guru e DigitalOcean foram usados apenas como apoio secundário para validação de descrições comuns de padrões, sem reprodução de conteúdo.